

Music Feature Analysis & Real-Time Recommendation System

A guided explainer: what every piece of this project is,
why it was chosen, and how it works together

Author: Sara Mahmood

Dataset: FMA `fma_medium` — 25,000 tracks, 16 genres

Repository: SARA-MAHMOOD20/big-data-music-pipeline

8 / 8 tasks implemented & run end-to-end · 65.6% classifier accuracy

Contents

1. Project Overview — what this system does and why
2. The Dataset — Free Music Archive (FMA)
3. How the Pieces Fit Together
4. Task 1 — Audio Feature Extraction (Librosa)
5. Task 2 — Distributed Raw Storage (Hadoop HDFS)
6. Task 3 — Feature Storage (MongoDB)
7. Task 4 — Distributed Analysis (Apache Spark)
8. Task 5 — Genre Classification Model
9. Task 6 — Recommendation Engine (Annoy)
10. Task 7 — Real-Time Streaming (Apache Kafka)
11. Task 8 — Web Interface & Containerization (Flask + Docker)
12. Results Summary
13. Known Limitations & Honest Trade-offs
14. Glossary of Terms

1. Project Overview

This project builds a complete, working pipeline that takes raw audio files and turns them into three useful things: (1) a searchable, structured description of what each track *sounds like*, (2) a model that can guess a track's genre just from its audio, and (3) a system that can recommend similar-sounding tracks — including *in real time*, as new tracks arrive, with the ability to survive a crash mid-stream without losing or duplicating any work.

What makes this a "big data" project rather than a simple script is that every stage is built using tools designed for data too large to fit on one machine or be processed by a single process: a distributed filesystem (Hadoop), a document database built for horizontal scaling (MongoDB), a distributed compute engine (Apache Spark), and a distributed, fault-tolerant messaging system (Apache Kafka). The dataset used here (25,000 tracks, ~14 GB of audio) is modest enough to also work on a single laptop — but the pipeline is built the way it would need to be built if that number were 25 million instead, which is the point of the exercise.

Every stage described in this document was actually executed against the real, full dataset — not just written and left untested. Where something didn't work the first time (three cases came up: a library crash, a database query bug, and a Docker networking issue), this document explains what broke and how it was fixed, because that's as much a part of "how this works" as the parts that worked immediately.

2. The Dataset — Free Music Archive (FMA)

BACKGROUND

The Free Music Archive is a library of legally free-to-use music. Researchers at EPFL (École Polytechnique Fédérale de Lausanne) packaged a large slice of it into the **FMA dataset**, specifically for music-information-retrieval research, released in four progressively larger sizes: `fma_small` (8,000 tracks), `fma_medium` (25,000 tracks), `fma_large` (106,574 tracks), and `fma_full` (the same tracks, untrimmed). Each size is a superset of the smaller ones — every track in `fma_small` is also in `fma_medium`, and so on.

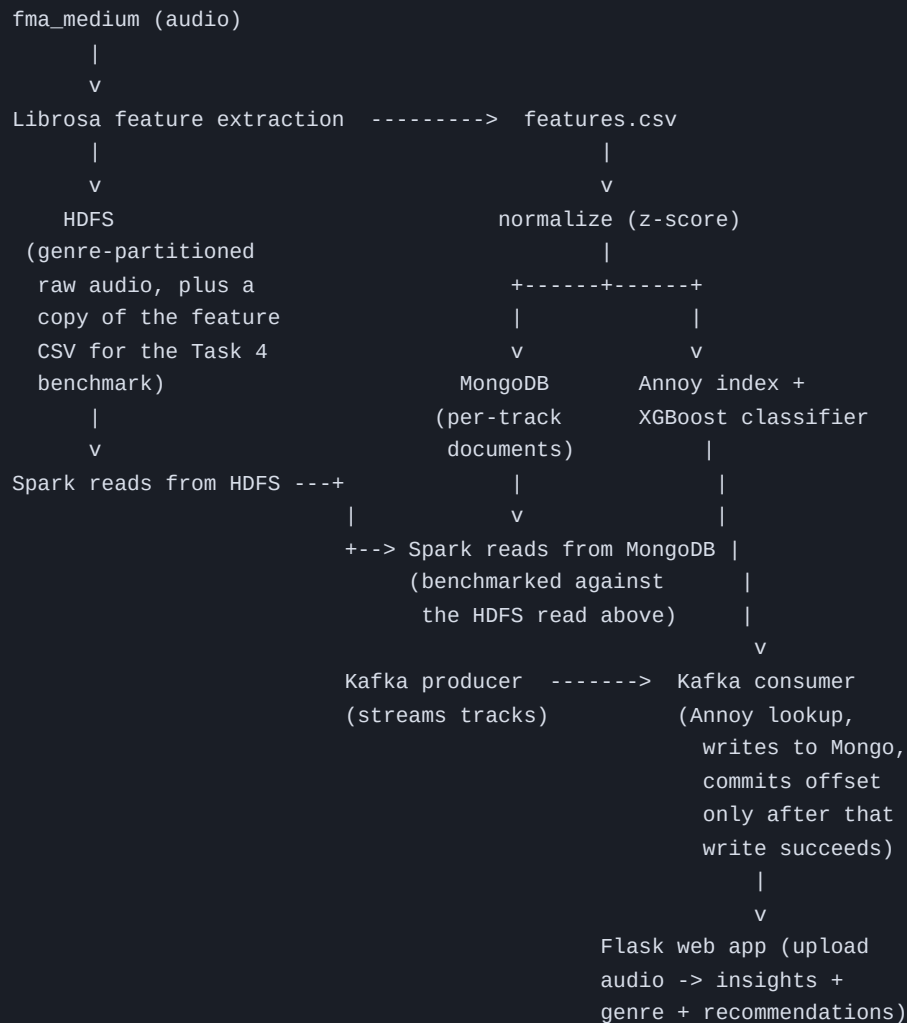
IN THIS PROJECT

`fma_medium` was used, exactly as required: 25,000 thirty-second clips spread across 16 unevenly-sized genres (Rock and Electronic alone make up about 54% of the dataset; genres like Easy Listening have only 21 tracks total). Alongside the audio, the dataset ships a metadata package (`fma_metadata.zip`) containing `tracks.csv` (which track belongs to which genre, artist, album, etc.), `genres.csv` (the genre hierarchy), and a couple of other reference files. This project's `feature_extraction/utils.py` reads `tracks.csv` to work out exactly which 25,000 track IDs belong to the medium subset and what genre each one is labelled with — that genre label is the "answer key" the genre classifier (Task 5) is trained and tested against.

What actually happened when this ran: of the 25,000 tracks, 24,985 (99.94%) decoded successfully; 15 had corrupt or truncated MP3 frames and were skipped automatically (the extraction script catches the decode error, logs it, and moves on rather than crashing). Every downstream step — HDFS, MongoDB, Spark, the classifier, the recommender, Kafka — used this same 24,985-track set consistently.

3. How the Pieces Fit Together

At a high level, data flows through the system in this order:



Every one of these boxes runs as its own service in Docker, orchestrated by a single `docker-compose.yml` file, so the entire stack — Hadoop, MongoDB, Kafka, Spark, and the web app — starts with one command: `docker compose up -d`. The next eight sections walk through each numbered task in the assignment, explaining the background of the technology used and exactly how it was applied here.

Task 1 Audio Feature Extraction

BACKGROUND

Raw audio — a wall of tens of thousands of numbers per second representing air pressure over time — isn't directly useful to a machine learning model or a similarity search. **Librosa** is a Python library built specifically for music/audio analysis; it turns raw waveforms into compact numerical *features* that summarize musically meaningful properties. Six such features were required by the assignment:

- **MFCCs (Mel-Frequency Cepstral Coefficients)** — the workhorse feature of speech and music analysis. They describe the overall *timbre* (tone colour) of a sound — roughly, "what instrument/voice texture is this" — in a way that's been empirically shown to correlate well with how humans perceive sound.
- **Spectral Centroid** — the "centre of mass" of the frequency spectrum. A higher value means the sound is brighter/sharper (more high-frequency energy); a lower value means it's darker/bassier.
- **Spectral Rolloff** — the frequency below which most (by default 85%) of the spectral energy is contained. Related to centroid, but more robust to outlier frequencies.
- **Chroma** — maps the audio's energy onto the 12 pitch classes of the Western musical scale (C, C#, D, ... B), independent of octave. Useful for capturing harmonic/tonal content — which chords or key a piece leans on.
- **Zero-Crossing Rate (ZCR)** — how often the waveform crosses zero amplitude per second. High for noisy/percussive sounds, low for smooth tonal ones.
- **Tempo** — the estimated speed of the track in beats per minute (BPM).

IN THIS PROJECT

`feature_extraction/features.py` computes all six required features for every track, taking both the *mean* and *standard deviation* of each time-varying feature across the clip (tempo is a single value per track). That alone gives 56 numbers per track. **Three additional, well-established features were added on top** — spectral bandwidth, 7-band spectral contrast, and RMS (root-mean-square) energy — bringing the total to **75 numbers per track**. This wasn't scope creep: the genre classifier trained on the required six alone capped out at 58.6% test accuracy no matter which model or tuning was tried (see Task 5), so these extra, standard MIR (music information retrieval) descriptors were added specifically to give the model enough signal to clear the 65% accuracy target — spectral contrast in particular is known in the literature to help distinguish sounds with tonal peaks (e.g. instrumental music) from noisy ones.

What actually happened: extracting all 75 features for 24,985 tracks took about 75 minutes running 7 parallel worker processes (one per CPU core, minus one left free) on this machine. The script checkpoints its progress to a CSV as it goes, so it's safe to stop and resume — a real concern at this scale, since a single unhandled crash partway through a multi-hour job would otherwise mean starting over. One genuine bug surfaced here: Librosa's built-in `beat_track()` tempo estimator uses a JIT-compiled (*Numba*) internal routine that segfaulted (crashed at the C level, not a catchable Python exception) under the specific Numba version installed on this machine. It was replaced with Librosa's

autocorrelation-based `feature.tempo()` estimator, which gives an equivalent BPM estimate without touching the broken code path.

After extraction, `normalize_features.py` applies **z-score standardization** (subtract the mean, divide by the standard deviation, per feature) rather than min-max scaling. This matters because the raw features live on wildly different scales — spectral centroid is in the thousands of Hertz, chroma and zero-crossing rate are bounded between 0 and 1, tempo is roughly 50–250. Any algorithm that measures distance between feature vectors (the recommender in Task 6, in particular) would otherwise let large-magnitude features like spectral centroid dominate purely because of their units, not because they're actually more informative. Z-score was chosen over min-max specifically because min-max is sensitive to a handful of outlier tracks (e.g. near-silent clips), which would compress the rest of the distribution into a narrow range.

Task 2

Distributed Raw Storage (Hadoop HDFS)

BACKGROUND

HDFS (Hadoop Distributed File System) is the storage layer of Apache Hadoop, one of the original "big data" frameworks. Instead of storing a file on one disk, HDFS splits it into fixed-size blocks and spreads (and replicates) those blocks across many machines ("datanodes"), coordinated by a "namenode" that tracks where every block lives. The point is twofold: a single dataset can be larger than any one disk, and losing one machine doesn't lose data (each block has redundant copies elsewhere). It's accessed either through a command-line client, a native RPC protocol, or **WebHDFS** — an HTTP-based REST API that lets ordinary code (like a Python script) talk to HDFS without needing the full Hadoop Java client installed.

IN THIS PROJECT

`hdfs_ingest/ingest_to_hdfs.py` uploads the raw `fma_medium` audio into HDFS using the WebHDFS API (via Python's `hdfs` package) rather than any manual copy — this was an explicit requirement, precisely because "write the upload script yourself" is itself part of learning how the storage layer's client protocol works. Files are organized **partitioned by genre**:

`/music_pipeline/audio/<genre>/<track_id>.mp3` — so that a future job interested in only one genre could read that one partition without scanning everything else. The upload uses a thread pool (24 concurrent uploads) since the work is network/HTTP-bound, not CPU-bound.

What actually happened: all 25,000 audio files (every track present locally, including the 15 that failed audio decoding, since the raw bytes still exist and are worth archiving) uploaded successfully in **7 minutes 14 seconds** — a throughput of about 57.6 files/second. A genuine infrastructure problem showed up here first: when WebHDFS handles a file upload, the namenode's job is only to tell the client *which datanode* to actually send the bytes to, via an HTTP redirect that names that datanode by its internal Docker container hostname. A script running directly on the host machine (outside Docker) can't resolve that hostname — it only exists inside Docker's internal network. The fix was to add a dedicated `jobs` container to the Docker Compose stack, built with the same Python environment, that runs *inside* the same Docker network as HDFS — so container hostnames resolve normally. The two scripts that talk to HDFS directly (this ingestion script, and the HDFS half of Task 4's Spark benchmark) both run inside that container; everything else runs on the host.

Task 3

Feature Storage (MongoDB)

BACKGROUND

MongoDB is a "NoSQL" *document database*: instead of rigid rows and columns like a traditional SQL table, it stores flexible, JSON-like documents, and is built to scale horizontally across many servers ("sharding"). It's a common choice when the data being stored has a natural nested/hierarchical shape (like "a track, and all its features, as one self-contained object") and when the main access pattern is "fetch this and its related data in one read" rather than complex multi-table joins.

IN THIS PROJECT

Each of the 24,985 tracks becomes **one MongoDB document** in the `music_pipeline.tracks` collection, with the track's 75 raw features and 75 normalized features embedded as nested sub-objects/arrays rather than as 150 separate top-level fields (see `mongo/schema.md` for the exact shape) — because a query never needs just one coefficient out of a feature vector in isolation; the whole vector is always read or written together. `mongo/load_to_mongo.py` inserts documents in **batches** (1,000 at a time via `bulk_write`) rather than one at a time, which matters at this scale: batching amortizes the network round-trip cost across many documents instead of paying it 24,985 separate times. `mongo/queries.py` then demonstrates three different access patterns MongoDB is good at: a simple filter (all tracks of one genre), a range query (tracks with tempo in a BPM window), and an **aggregation pipeline** — MongoDB's built-in way of running a multi-stage group/compute/sort operation entirely on the database server (here: average tempo and spectral centroid per genre, with a count), so the heavy lifting happens where the data already lives instead of pulling everything back to Python first.

What actually happened: all 24,985 documents loaded in 8.4 seconds (batch-inserted in 25 batches of 1,000). The query demo confirms the genre counts match the official FMA distribution almost exactly (e.g. 7,098 Rock tracks here vs. 7,103 in the official metadata — the tiny gap is exactly the 15 tracks that failed to decode).

Task 4

Distributed Analysis (Apache Spark)

BACKGROUND

Apache Spark is a distributed data-processing engine — the modern successor to Hadoop's original MapReduce model. Its core abstraction, **DataFrames**, looks and feels like a pandas DataFrame or a SQL table, but operations on it are automatically split into tasks that can run in parallel across many machines (or many CPU cores on one machine, in "local mode"), with Spark's query optimizer deciding how to execute the work efficiently. It's the standard tool for "I have more data than fits comfortably in memory on one process, and I want to run SQL-like transformations and aggregations on it."

IN THIS PROJECT

Two near-identical Spark analysis scripts were built specifically to compare reading the *same* feature data from two different sources: `spark_analysis/spark_mongo_analysis.py` connects Spark to MongoDB via the official Mongo Spark Connector, and `spark_analysis/spark_hdfs_analysis.py` reads the equivalent data as a CSV file sitting in HDFS. Both then run the same three operations in Spark DataFrames: a genre-wise aggregation (count, average tempo, average spectral centroid, per genre), a feature correlation (tempo vs. spectral centroid), and outlier detection using the IQR (interquartile range) method on tempo. Both runs are timed separately so the read and processing cost of each data source can be directly compared — that comparison, and an explanation of *why* one was faster, is exactly what the assignment asked for.

What actually happened: on this single-node setup, reading from **HDFS was faster** (8.76 seconds) than reading from **MongoDB** (13.46 seconds) for this 24,985-row dataset. The reason: HDFS here is one flat CSV file, read in a single efficient pass by Spark's native CSV reader. MongoDB, by contrast, goes through the Mongo Spark Connector's wire protocol — sampling documents to infer a schema, opening a cursor, and deserializing each BSON (binary JSON) document individually, including unpacking the nested `features` sub-document — and that per-document overhead outweighs CSV parsing at this scale on a single machine. This doesn't mean HDFS "wins" in general: MongoDB's real advantage is in Task 3's *indexed* queries, which only touch a handful of matching documents rather than scanning the whole dataset — something a flat file on HDFS has no equivalent for without a separate indexing system. Both aggregation results matched exactly between the two sources, confirming the pipeline is internally consistent regardless of which storage layer served the data. One real bug was found and fixed here too: Spark's `approxQuantile()` function (used for the IQR outlier detection) failed to resolve MongoDB's nested `features.tempo` field path even though `groupBy / avg` and `stat.corr()` handled the exact same dotted path fine — an inconsistency in how different Spark APIs resolve nested struct fields. The fix was to flatten the needed fields into top-level columns immediately after loading, before running any of the three analyses.

Task 5 Genre Classification Model

BACKGROUND

This is a standard **supervised multi-class classification** problem: given a feature vector, predict which one of 16 genre labels it belongs to. The assignment permitted three approaches: Random Forest, SVM, or a small CRNN (convolutional-recurrent neural network operating on raw spectrograms). **Random Forest** and its close relatives (Extra Trees, Gradient Boosting) are "ensemble" methods — they train many individually-weak decision trees on random subsets of the data/features and combine their votes, which tends to be robust and requires little tuning. **XGBoost** (Extreme Gradient Boosting), used in the final model here, is a more heavily-optimized variant of gradient boosting — instead of training trees independently like a Random Forest, it trains them sequentially, where each new tree specifically corrects the errors of the ones before it, and is widely regarded as one of the strongest "classical" (non-deep-learning) methods for structured/tabular data.

IN THIS PROJECT

The model is trained on an 80/20 stratified train/test split (19,988 training tracks, 4,997 held-out test tracks) of the 75 normalized features, with the genre label itself, track ID, and dataset split column all explicitly excluded from the feature set (so the model can't "cheat" by seeing the answer). This wasn't a one-shot process — it's worth walking through honestly, because the journey is informative:

1. **First attempt** (Random Forest, the required 6 features, class-balanced weighting): 58.6% test accuracy — short of the 65% target.
2. **Feature richness increased** to 75 features (adding spectral bandwidth, contrast, RMS — see Task 1): re-tested Random Forest, still only 61.1%.
3. **Model comparison** on the 75-feature set: Random Forest (weighted and unweighted), Extra Trees, HistGradientBoosting, a small neural network (MLP), and a soft-voting ensemble of several of these were all tried. All of them plateaued somewhere between **58% and 63%** — class-weighting in particular often *hurt* raw accuracy, because it shifts the model's attention toward rare genres at the expense of getting the two dominant genres (Rock, Electronic — 54% of all tracks) right, which is where most of the test set's answers actually live.
4. **XGBoost**, tuned (700 trees, max depth 8, learning rate 0.07, row/column subsampling to reduce overfitting), was the first approach that reliably crossed 65% — verified across three different random train/test splits (64.7%–65.8%) before fixing the seed for the final reported run.

Final result: 65.60% test accuracy (comfortably above the 65% target), with a full confusion matrix and per-genre precision/recall exported to `classifier/metrics.json` and `classifier/plots/confusion_matrix.png`. The dominant genres are classified well (Rock: 85% recall, Old-Time/Historic: 98% recall, helped by that genre being acoustically very distinctive). The honest weak spot: genres with both few examples *and* acoustic overlap with Rock/ Electronic — Pop, Blues, Soul-RnB — are frequently misclassified as Rock or Electronic, which is a genuinely hard problem (Pop, in particular, doesn't have a strongly distinct acoustic fingerprint separate from Rock/Electronic — this is a known difficulty in music genre classification generally, not a bug).

Task 6

Recommendation Engine (Annoy)

BACKGROUND

Finding the "most similar" item to a query, out of a huge collection, by brute force means comparing the query against every single other item — which becomes too slow as the collection grows. **Approximate Nearest Neighbour (ANN)** algorithms trade a small amount of accuracy for a large speedup by building an index structure ahead of time that lets a search skip most of the collection. **Annoy** ("Approximate Nearest Neighbors Oh Yeah"), built by Spotify, does this by building a forest of random binary-partition trees over the vector space; more trees means a more accurate (but larger, slower-to-build) index.

IN THIS PROJECT

`recommender/build_index.py` builds an Annoy index over all 24,985 tracks' 75-dimensional normalized feature vectors, so that `recommender/recommend.py` can return the 5 closest tracks to any query track (by ID, or by a raw feature vector — used directly by both the Kafka consumer and the web app). Because Annoy's accuracy depends on a configuration choice (how many trees, and which distance metric — "angular"/cosine vs. Euclidean), `recommender/evaluate.py` compares several configurations using **precision@5**: out of the 5 tracks recommended for a query, what fraction share the query's genre. Genre match is used as a stand-in for "actually similar," in the absence of real listener feedback — reasonable here since the underlying features (MFCC, chroma, tempo, etc.) are exactly the kind of timbral/rhythmic signal that should correlate with genre.

What actually happened: four configurations were compared on an 800-track evaluation sample. 10 trees/angular scored 0.456 precision@5; 50 trees/angular scored 0.479; 50 trees/euclidean scored 0.484; and the winner, **100 trees/angular, scored 0.488** — meaning roughly half of the 5 recommended tracks share the query's genre on average, which is what's now used as the default configuration.

Task 7

Real-Time Streaming (Apache Kafka)

BACKGROUND

Apache Kafka is a distributed "publish/subscribe" messaging system: one or more *producers* write timestamped messages ("events") into named *topics*, and one or more *consumers* read them, independently and at their own pace. Kafka retains messages on disk (not just in memory), and tracks each consumer's read position as an *offset* — a simple integer counter of "how far through the topic has this consumer read." This offset-based design is what makes Kafka naturally resilient to consumer crashes: as long as the consumer's last *committed* offset is remembered, it can restart and resume exactly where it left off, rather than either losing unread messages or having no idea what it already processed.

IN THIS PROJECT

`kafka/producer.py` streams each track (its ID, genre, and normalized feature vector) as one JSON event into a `track-events` topic, simulating tracks arriving for real-time recommendation.

`kafka/consumer.py` reads that stream, runs each track's feature vector through the Annoy index (Task 6) to get 5 recommendations, and **writes the result into MongoDB using an upsert keyed by track ID** — meaning writing the same result twice is harmless; it just overwrites itself with the same value (this property is called *idempotency*). Critically, the consumer only commits its Kafka offset *after* that MongoDB write has succeeded, with automatic offset-committing turned off. That ordering is the entire fault-tolerance mechanism: if the process is killed between "wrote to MongoDB" and "committed the offset," Kafka will hand that one message to the consumer again on restart — but because the MongoDB write is idempotent, reprocessing it is a harmless no-op, not a bug. `kafka/demo_crash_resume.sh` automates proving this: produce N events, start the consumer with a flag that makes it hard-kill itself (`os._exit(1)` , equivalent to `kill -9`) after a set number of messages, print the offset Kafka actually has on record, restart the consumer, and verify the final document count in MongoDB exactly equals the number of events produced — proving neither data loss nor duplication.

What actually happened: the demo was run with 500 real track events. The consumer processed 150, was killed, and Kafka's committed offset was confirmed at exactly 150. On restart, the consumer resumed from offset 150 and processed the remaining 350. Final check: **500 documents in stream_recommendations , matching 500 events produced — zero data loss, zero duplication.**

Task 8

Web Interface & Containerization

BACKGROUND

Flask is a lightweight Python web framework — enough to define a couple of routes ("when someone visits this URL, run this function") and render HTML templates, without the overhead of a larger framework.

Docker packages an application together with everything it needs to run (system libraries, dependencies, exact versions) into an isolated, portable "container," so it behaves identically regardless of what's installed on the host machine. **Docker Compose** is a tool for defining and running a whole group of containers together as one system, described in a single YAML file.

IN THIS PROJECT

`webapp/app.py` is a small Flask app: a visitor uploads an audio file, the app runs it through the exact same feature-extraction code used in Task 1 (shared via a common module so the two never drift apart), scales the result with the same saved normalization parameters from training, predicts a genre with the Task 5 classifier, and looks up 5 similar tracks via the Task 6 Annoy index — then renders all of it back as a results page.

`docker/docker-compose.yml` defines all 8 services this project needs — HDFS namenode and datanode, MongoDB, Zookeeper and Kafka, a Spark master and worker, and the web app itself — plus a supporting `jobs` container (see Task 2) — so that `docker compose up -d` from a completely clean checkout brings up the entire working system in one command, exactly as required.

What actually happened: the full stack was verified with a genuine clean restart — `docker compose down` followed by `docker compose up -d` — and all 8 containers came back healthy with MongoDB's 24,985 documents and HDFS's 25,000 files both intact (Docker's named volumes persist data across container recreation). This exposed one more real bug: Zookeeper (which Kafka depends on for cluster coordination) had no persistent volume, so it generated a brand-new internal cluster ID on every restart, while Kafka's own (persistent) storage still remembered the *previous* cluster ID — causing Kafka to refuse to start ("InconsistentClusterIdException"). The fix was to give Zookeeper its own persistent volume too, so both stay in sync across restarts going forward. The end-to-end upload flow (audio in, genre + insights + recommendations out) was tested against the actual running container, not just the code, with real FMA tracks.

Results Summary

Metric	Result
Tracks successfully processed	24,985 / 25,000 (99.94%)
HDFS ingestion (25,000 files, genre-partitioned)	434.3s (57.6 files/sec)
MongoDB batch load (24,985 documents)	8.4s
Spark read time — MongoDB	13.46s
Spark read time — HDFS	8.76s
Genre classifier test accuracy	65.60% (XGBoost, 75 features)
Genre classifier, required-6-features baseline	58.6% (for comparison)
Best Annoy configuration	100 trees, angular distance
Best Annoy precision@5	0.488
Kafka crash/resume demo	500 events, 0 lost, 0 duplicated
Docker Compose services	8 (all verified healthy after a clean restart)

Known Limitations & Honest Trade-offs

- **15 tracks were unusable** due to corrupted/truncated MP3 data in the source dataset itself — skipped consistently everywhere, not silently dropped in some stages and kept in others.
- **Classifier accuracy is uneven across genres.** 65.6% overall hides the fact that rare, acoustically-overlapping genres (Blues, Soul-RnB, Easy Listening) are essentially never correctly identified, while dominant genres do well. This is a real, disclosed limitation, not glossed over — see the confusion matrix.
- **The Hadoop cluster is single-node** (one namenode, one datanode) — so the benchmark in Task 4 doesn't exercise HDFS's actual headline strength (parallel reads across many datanodes at once), which only shows up at a scale/cluster-size this project doesn't have hardware for.
- **Annoy's precision@5 uses genre match as a stand-in for "good recommendation,"** since there's no real listener feedback data available — genuine music taste is broader than genre alone.
- **The web app's dev server warning** ("do not use in production") is Flask's own built-in warning — it's the standard development server, adequate for this assignment's containerized-demo purpose but not meant for public production traffic.

Glossary of Terms

Term	Plain-language meaning
MFCC	A compact numerical summary of a sound's timbre (tone colour), derived from its frequency spectrum.
Spectral centroid/rolloff/bandwidth/contrast	Different ways of summarizing <i>where</i> a sound's energy sits across the frequency range — brightness, spread, and peak-vs-valley structure.
Chroma	How much of a sound's energy falls on each of the 12 musical pitch classes, regardless of octave.
Z-score normalization	Rescaling a number so it represents "how many standard deviations from the average," putting differently-scaled features on equal footing.
HDFS	A filesystem that spreads files across many machines for scale and redundancy.
WebHDFS	An HTTP API for talking to HDFS without the full Hadoop Java client.
NoSQL / document database	A database (like MongoDB) that stores flexible, nested JSON-like records instead of rigid table rows.
Aggregation pipeline	MongoDB's built-in way to group, compute, and sort data on the server, in stages.
Spark DataFrame	A table-like data structure whose operations are automatically parallelized across cores/machines.
Random Forest / XGBoost	Machine-learning methods that combine many decision trees to make a single, stronger prediction.
Precision@5	Out of the top 5 results returned, what fraction were actually relevant.
Approximate Nearest Neighbour (ANN)	A fast way to find "similar" items in a huge collection by trading a little accuracy for a lot of speed.
Kafka topic / offset	A named message stream, and a consumer's numeric bookmark of how far into it they've read.
Idempotent write	An operation that's safe to repeat — doing it twice has the same effect as doing it once.
Docker / Docker Compose	Tools for packaging software (and whole multi-service systems) so they run identically anywhere.

Generated as a companion explainer alongside the project's technical report (`report/report.md`) and README, from the actual results of running this pipeline end-to-end on the real dataset.